Analyse d'images

Comptes-rendus de Travaux Pratiques

4e année IMI — CPE Lyon

TPs couverts

- ✓ **TP 1 — Détection d'objets par Deep Learning**
- ✓ **TP 2 — Classification d'images avec CIFAR-10**
- ✓ **TP 3 — Suivi de balle par couleur (OpenCV)**

Paul-Malo Poisson

CPE LYON

Contents

I TP Détection — Détection d'objets par Deep Learning 4

1 Introduction 4

2 Partie 1 : Prise en main de YOLO avec OpenCV 4
2.1 Classes reconnues 4
2.2 Rôle des seuils de détection 4
2.3 Architecture des réseaux 5

3 Partie 2 : Interfaçage Python 5
3.1 Filtrage par classe et flux webcam 5
3.2 Sauvegarde et mosaïque des bounding boxes 5

4 Partie 3 : Entraînement d'un détecteur personnalisé 5
4.1 Constitution du dataset avec Roboflow 6
4.2 Entraînement avec Ultralytics et export ONNX 6

5 Détection d'objets avec TensorFlow 6
5.1 Classification, détection et segmentation 6
5.2 Panorama des architectures de détection 6
5.3 Classes et architectures des modèles TensorFlow Hub 7
5.4 Comparaison des modèles 7

6 TensorFlow Hub et plateformes modernes 7
6.1 Rôle et évolution de TensorFlow Hub 7
6.2 Familles d'architectures disponibles 8
6.3 Fine-tuning 8

7 Architecture des réseaux de détection 8
7.1 Format d'entrée 8
7.2 Du classifieur au détecteur 8
7.3 Phase d'apprentissage 9

8 Solutions modernes : kitt.tools et Transformers 9
8.1 kitt.tools 9
8.2 Utilisations étendues des modèles modernes 9

9 Conclusion — TP Détection 9

II TP Classification — Classification d'images avec CIFAR-10 10

10 Introduction 10

11 Paramétrage et résultats expérimentaux 10

11.1 Configuration de l'entraînement 10
11.2 Comparaison des architectures 11
11.3 Analyse de la généralisation et de l'overfitting 11
11.4 Matrice de confusion et types d'erreurs 11

12 Anatomie d'un réseau convolutif **11**
12.1 Couches de convolution 11
12.2 Couches de pooling et régularisation 12
12.3 Fonctions d'activation 12

13 Pistes d'amélioration **12**

14 Synthèse des résultats **12**
14.1 L'avantage décisif des CNN 13
14.2 Overfitting et hyperparamsètres 13

15 Explicabilité via Grad-CAM++ **13**
15.1 Principe 13
15.2 Observations sur notre modèle 13
15.3 Autres méthodes d'explicabilité 13

16 Généralisation et enrichissement des datasets **14**
16.1 Application sur d'autres datasets 14
16.2 Enrichissement de CIFAR-10 14
16.3 Enrichissement de Fashion MNIST 14

17 Conclusion — TP Classification **14**

III TP Couleur — Suivi de balle par couleur avec OpenCV **16**

18 Introduction **16**

19 Question 1 — Analyse du code `balls_tracker.py` **16**
19.1 Pipeline de traitement 16
19.2 Utilisation comme capteur de position 17

20 Question 2 — Suivi multi-couleur **17**
20.1 Plages HSV utilisées 17
20.2 Code proposé 17
20.3 Résultats 18

21 Question 3 — Contrôle de drone (`autopylot_agent.py`) **18**
21.1 Architecture du système 18
21.2 Calcul des erreurs et contrôle proportionnel 18

22 Question 4 — Deux balles de même couleur **18**
22.1 Comportement observé 18
22.2 Conséquences et solution 19

23 Question 5 — Suivi du plus grand contour **19**

24 Question 6 — Plus grosse balle rose (`ball4.mp4`) 19

25 Questions 8 & 9 — Persistance du suivi 20
25.1 Le problème du saut de cible (Q8) 20
25.2 Suivi par proximité spatiale (Q9) 20

26 Question 10 — Apprentissage automatique de la couleur 21
26.1 Principe 21

27 Question 11 — Estimation de distance par interpolation 21
27.1 Modèle théorique 21
27.2 Calibration empirique 21

28 Question 12 — Estimation de distance par calibration caméra 22
28.1 Modèle sténopé 22
28.2 Comparaison des deux approches 22

29 Conclusion — TP Couleur 23

Part I

TP Détection — Détection d'objets par Deep Learning

1 Introduction

Ce TP aborde la détection d'objets par apprentissage profond, une tâche plus exigeante que la simple classification puisqu'elle impose de *localiser* les objets dans l'image en plus de les nommer. Cette localisation passe par la prédiction de rectangles englobants (*bounding boxes*) associés à un label et à un score de confiance.

Plusieurs outils ont été mis en œuvre au fil des manipulations : YOLOv3 et YOLOv3-Tiny via le module `cv2.dnn` d'OpenCV pour la détection en temps réel, TensorFlow Hub pour l'étude d'architectures modernes pré-entraînées, et la chaîne Roboflow/Ultralytics pour la création et l'entraînement d'un détecteur personnalisé. Le format ONNX a également été utilisé comme vecteur d'interopérabilité entre frameworks.

2 Partie 1 : Prise en main de YOLO avec OpenCV

2.1 Classes reconnues

Les modèles YOLOv3 utilisés dans ce TP sont entraînés sur COCO (*Common Objects in Context*), un dataset de référence comportant 80 catégories couvrant un large éventail d'objets du quotidien : personnes, véhicules, animaux, électronique, mobilier, etc. La liste complète est chargée depuis le fichier `coco.names` au démarrage du réseau.

Les tests menés sur plusieurs images révèlent une différence notable entre les deux variantes : YOLOv3 détecte davantage d'objets avec une meilleure précision, tandis que YOLOv3-Tiny, de par son architecture allégée, privilégie la vitesse d'exécution au détriment d'une légère perte de précision.

2.2 Rôle des seuils de détection

Deux seuils pilotent la qualité des détections produites.

Le **confidence threshold** fixe le score minimal en dessous duquel une détection est rejetée. Abaisser ce seuil augmente la sensibilité du détecteur mais au prix d'un nombre plus élevé de faux positifs ; le relever produit l'effet inverse.

Le **NMS threshold** (*Non-Maximum Suppression*) résout le problème des boîtes multiples générées pour un même objet. L'algorithme conserve la boîte de plus haut score et supprime toutes celles dont le chevauchement (IoU) dépasse le seuil fixé. Le réglage conjoint de ces deux paramètres est donc déterminant pour la lisibilité des résultats.

2.3 Architecture des réseaux

YOLOv3 adopte le paradigme *single-shot* : l'image entière est traitée en une seule passe du réseau, ce qui garantit une latence très faible. L'architecture s'appuie sur un empilement de couches convolutives suivi de têtes de détection opérant à trois échelles différentes, ce qui lui permet de repérer aussi bien les grands que les petits objets.

YOLOv3-Tiny est une version élaguée : le nombre de couches et de paramètres est fortement réduit, ce qui accélère l'inférence mais restreint la capacité d'extraction de caractéristiques. Un point important : les dernières couches du réseau dépendent directement du nombre de classes cibles, et doivent donc être reconfigurées lors de tout entraînement sur un dataset personnalisé.

3 Partie 2 : Interfaçage Python

3.1 Filtrage par classe et flux webcam

Une fonctionnalité de filtrage a été intégrée au programme de base, permettant de restreindre les détections affichées à un sous-ensemble de classes définies par l'utilisateur (par exemple uniquement les personnes ou les véhicules). Ce filtrage est particulièrement utile dans les cas applicatifs où seule une catégorie d'objets est pertinente.

Le programme gère également l'acquisition depuis une webcam. Les tests réalisés montrent que YOLOv3-Tiny maintient un débit d'images satisfaisant pour une utilisation temps réel sur matériel standard, là où YOLOv3 complet induit une latence plus perceptible.

Les résultats de chaque frame sont exportés dans un fichier JSON contenant, pour chaque détection, le nom de la classe, le score de confiance et les coordonnées de la bounding box. Ce format universel facilite la transmission des données vers d'autres modules ou applications.

3.2 Sauvegarde et mosaïque des bounding boxes

Chaque région détectée est découpée de l'image source et enregistrée individuellement en JPEG. L'ensemble des découpes appartenant à une même classe est ensuite assemblé en mosaïque, ce qui offre une vue synthétique de la qualité du détecteur et constitue une base de données prête à l'emploi pour de futurs entraînements.

4 Partie 3 : Entraînement d'un détecteur personnalisé

4.1 Constitution du dataset avec Roboflow

La première étape a consisté à collecter des photos originales des objets cibles sous diverses conditions d'éclairage et d'arrière-plan, puis à les annoter manuellement via l'interface Roboflow. Cet outil simplifie considérablement le pipeline d'annotation gr^ace à une interface web ergonomique, et génère automatiquement les fichiers de configuration et les divisions train/val/test.

4.2 Entraînement avec Ultralytics et export ONNX

Le dataset exporté au format YOLOv5 a ensuite été utilisé avec la librairie Ultralytics sur Google Colab. Les résultats obtenus sur notre dataset personnalisé surpassent ceux de l'entraînement natif Roboflow, ce qui s'explique par le meilleur contrôle des hyperparamsètres et la durée d'entraînement plus longue disponible sur GPU.

Le modèle final a été exporté au format **ONNX**, standard ouvert qui permet de déployer un même réseau sur des frameworks variés (OpenCV DNN, TensorRT, ONNX Runtime, etc.) sans reconversion. Le script de détection a été adapté pour charger ce modèle ONNX, ajuster les dimensions d'entrée et appliquer le post-traitement correspondant.

5 Détection d'objets avec TensorFlow

5.1 Classification, détection et segmentation

Ces trois tâches forment une hiérarchie de complexité croissante en vision par ordinateur.

La **classification** produit un unique label global pour l'image entière, sans aucune information spatiale sur la localisation des objets représentés.

La **détection** ajoute la dimension de localisation : chaque objet identifié est matérialisé par une bounding box associée à un label et à un score de confiance. C'est le régime d'opération des architectures YOLO et SSD étudiées dans ce TP.

La **segmentation** va encore plus loin en assignant une classe à chaque pixel individuel de l'image. En segmentation sémantique, tous les pixels d'une même catégorie reçoivent le même label ; en segmentation d'instances, chaque occurrence individuelle d'un objet possède son propre masque. Cette précision pixel-level est indispensable dans des domaines exigeants comme l'imagerie médicale ou les véhicules autonomes.

5.2 Panorama des architectures de détection

Le paysage de la détection d'objets par apprentissage profond est structuré autour de deux familles principales.

Les architectures **en une étape** (YOLO, SSD) traitent l'image en un seul passage réseau et produisent directement les boîtes et les labels. Leur atout majeur est la rapidité d'inférence. SSD se distingue par l'utilisation de cartes de caractéristiques multi-échelles permettant de couvrir des objets de tailles très différentes.

Les architectures **en deux étapes** (Faster R-CNN) décomposent le problème : un premier réseau (*Region Proposal Network*) génère des régions candidates, qu'un second réseau classifie ensuite. Cette séparation améliore la précision mais allonge le temps de calcul.

Des architectures plus récentes comme RetinaNet, EfficientDet ou DETR (*Detection Transformer*) cherchent à repousser ce compromis vitesse/précision en exploitant des mécanismes d'attention ou des pyramides de features bidirectionnelles.

5.3 Classes et architectures des modèles TensorFlow Hub

Les modèles disponibles sur TensorFlow Hub sont pour la plupart entraînés sur COCO, qui recense près de 90 classes couvrant personnes, véhicules, animaux, objets domestiques et électronique grand public.

Modèle	**Architecture**	**Caractéristique**
SSD MobileNetV2	Backbone MobileNetV2 + tête SSD multi-échelle	Léger, optimisé mobile/embarqué
EfficientDet	Backbone EfficientNet + BiFPN	Bon équilibre vitesse/précision
Faster R-CNN	Backbone Inception-ResNetv2 + RPN	Haute précision, plus lent
CenterNet	Hourglass Network	Détection par keypoints centraux

Table 1: Architectures des modèles TensorFlow Hub étudiés

5.4 Comparaison des modèles

SSD MobileNetV2 est le plus rapide mais peut manquer des objets dans des scènes complexes. EfficientDet offre un meilleur rappel global tout en restant exploitable en temps réel. Faster R-CNN décroche la meilleure précision mais son coût de calcul le destine avant tout à des applications où la latence n'est pas critique. Dans le domaine du temps réel contraint, les variantes récentes de YOLO (v5, v8, v11) demeurent la référence opérationnelle.

6 TensorFlow Hub et plateformes modernes

6.1 Rôle et évolution de TensorFlow Hub

TensorFlow Hub a été conçu par Google pour démocratiser l'accès aux modèles pré-entraînés : une seule ligne de code suffit pour charger un modèle complet avec ses poids. La plateforme a couvert la classification d'images, la détection d'objets, le traitement du langage naturel et la génération d'embeddings.

Aujourd'hui, l'écosystème s'est diversifié : Hugging Face Hub, Kaggle Models, Ultralytics Hub et ONNX Model Zoo proposent des catalogues plus étendus et souvent mieux maintenus, ce qui relègue TensorFlow Hub à un rôle secondaire dans les nouveaux projets.

6.2 Familles d'architectures disponibles

Ces plateformes hébergent trois grandes familles :

- architectures *single-stage* (YOLO, SSD) ;
- architectures *two-stage* (Faster R-CNN) ;
- architectures basées sur les Transformers (DETR).

Les modèles sont pré-entraînés sur COCO, ImageNet, Objects365 ou LVIS, certains de ces datasets atteignant plusieurs milliers de classes.

6.3 Fine-tuning

L'adaptation d'un modèle pré-entraîné à un dataset personnalisé exploite le *transfer learning* : le backbone conserve les représentations appris sur de grands corpus, tandis que la tête de détection est ré-entraînée sur les nouvelles données. Deux modes existent :

- **Fine-tuning partiel** : seule la tête est entraînée (backbone gelé).
- **Fine-tuning complet** : l'intégralité du réseau est ré-entraînée avec un faible taux d'apprentissage.

Dans les deux cas, les annotations doivent fournir des bounding boxes précises avec les labels correspondants.

7 Architecture des réseaux de détection

7.1 Format d'entrée

Les réseaux de détection attendent en entrée un tenseur 4D de la forme :

$$[\,\text{batch},\ \text{canaux},\ \text{hauteur},\ \text{largeur}\,]$$

YOLOv8n travaille typiquement avec des images 640×640 sur 3 canaux RGB, soit un tenseur $[1, 3, 640, 640]$. Les valeurs de pixels sont normalisées dans $[0, 1]$ avant d'entrer dans le réseau.

7.2 Du classifieur au détecteur

La conversion d'un classificateur CNN en détecteur requiert trois modifications principales :

1. suppression de la couche *fully-connected* finale qui produisait un vecteur de probabilités globales ;
2. ajout d'une tête de détection prédisant coordonnées des boîtes et scores de classes ;
3. intégration de mécanismes multi-échelles (*Feature Pyramid Networks*) pour couvrir des objets de tailles variées.

Faster R-CNN illustre ce principe : il greffe un RPN et une tête de classification sur un backbone ResNet50 pré-entraîné sur ImageNet.

7.3 Phase d'apprentissage

L'entraînement d'un détecteur diffère de celui d'un classificateur par la complexité de la fonction de perte, qui combine :

- une **perte de localisation** mesurant l'écart entre boîtes prédites et annotations (généralement IoU ou GIoU) ;
- une **perte de classification** pour les labels (entropie croisée).

Des techniques d'augmentation spécifiques à la détection sont également employées : retournement horizontal, recadrage aléatoire, mosaique d'images (mosaic augmentation), afin d'améliorer la robustesse aux variations de contexte.

8 Solutions modernes : kitt.tools et Transformers

8.1 kitt.tools

La plateforme `kitt.tools` propose un environnement de démonstration accessible directement depuis le navigateur, sans installation. Elle embarque plusieurs modèles spécialisés au format ONNX : DETR pour la détection, ViT (*Vision Transformer*) pour la classification, TrOCR pour la reconnaissance de texte, et CLIP pour l'alignement image-texte.

8.2 Utilisations étendues des modèles modernes

Ces architectures dépassent le cadre strict de la détection d'objets. CLIP permet par exemple d'effectuer de la recherche d'images par description textuelle ou du *zero-shot learning*. ViT peut être détourné pour l'extraction de features ou la segmentation sémantique. TrOCR traite la numérisation automatique de documents. DETR, enfin, prédit directement les bounding boxes via un mécanisme *encoder-decoder* Transformer, sans ancres prédéfinies (*anchor-free*).

9 Conclusion — TP Détection

Ce TP a permis d'appréhender la détection d'objets dans toute sa complexité, depuis la prise en main de modèles pré-entraînés jusqu'à la constitution et l'entraînement d'un détecteur sur mesure. Le format ONNX s'est révélé un maillon clé pour l'interopérabilité entre frameworks.

La comparaison des architectures (YOLO, SSD, Faster R-CNN, EfficientDet) met en évidence un compromis fondamental entre vitesse d'inférence et précision de détection : les variantes YOLO modernes restent la référence pour les applications temps réel, tandis que les approches deux-étapes s'imposent quand la précision prime. Les architectures Transformer (DETR) ouvrent de nouvelles perspectives en s'affranchissant des heuristiques d'ancres, au prix d'un entraînement plus coûteux.

Part II

TP Classification — Classification d'images avec CIFAR-10

10 Introduction

Les réseaux de neurones convolutifs (CNN) ont transformé l'analyse d'images en rendant possible une extraction automatique et hiérarchique de caractéristiques visuelles. Ce second TP se concentre sur la **classification d'images** à travers le dataset de référence **CIFAR-10** : 60 000 images couleur 32×32 pixels réparties en 10 catégories (avions, animaux, véhicules).

L'objectif central est d'étudier l'influence des choix architecturaux et des hyperparamsètres sur la capacité de généralisation des modèles, en distinguant rigoureusement les performances mesurées sur les données d'entraînement de celles obtenues sur le jeu de test.

Objectifs

- Quantifier l'impact du taux d'apprentissage, de la taille de lot et du nombre d'époques sur la convergence.
- Comprendre le rôle de chaque type de couche dans une architecture convolutive.
- Analyser l'overfitting à travers les courbes d'apprentissage et les matrices de confusion.
- Visualiser les zones d'attention du réseau via Grad-CAM++.
- Évaluer la généralisation sur d'autres datasets (MNIST, Fashion MNIST, CIFAR-100).

11 Paramétrage et résultats expérimentaux

11.1 Configuration de l'entraînement

L'optimiseur **Adam** avec un taux d'apprentissage initial de 0,001 a été retenu pour l'ensemble des expériences. La taille de lot est fixée à 256 images, et l'entraînement est poussé jusqu'à 100 époques pour rendre visible le phénomène de sur-apprentissage. La fonction de perte employée est la *Sparse Categorical Crossentropy*, adaptée aux labels entiers de CIFAR-10.

11.2 Comparaison des architectures

Architecture	Préc. Entr.	Préc. Test	Observation
Simplest NN	38,49%	35,47%	Sous-apprentissage
Vanilla NN (128)	43,52%	42,79%	Linéarité insuffisante
LeNet-5	61,56%	54,37%	Bonne base historique
Vanilla CNN	90,98%	73,93%	Architecture équilibrée
AlexNet	91,14%	80,59%	Risque d'overfitting

Table 2: Performances comparées des modèles sur CIFAR-10

La progression est nette : les modèles entièrement denses plafonnent sous les 45%, incapables de représenter les relations spatiales entre pixels. L'introduction des couches convolutives fait franchir le seuil des 70%, et AlexNet atteint 80,59% en test.

11.3 Analyse de la généralisation et de l'overfitting

L'écart entre précision d'entraînement et précision de test constitue le marqueur de l'overfitting. Pour le Vanilla CNN, cet écart dépasse 17 points ; pour AlexNet, bien que le score en test soit le meilleur, la différence reste importante. Ce phénomène s'explique par la mémorisation du bruit du jeu d'entraînement plutôt que par l'apprentissage de caractéristiques transférables. La précision en test demeure le seul indicateur valide de la performance réelle.

11.4 Matrice de confusion et types d'erreurs

L'examen des matrices de confusion révèle des tendances systématiques :

- **Chat / Chien** : forte confusion due à des caractéristiques visuelles partagées (oreilles, texture du pelage) difficiles à discriminer en 32×32 pixels.
- **Automobile / Camion** : formes géométriques similaires et contextes de fond communs (routes, asphalte).
- **Navire / Aéronef** : meilleurs scores, leurs arrière-plans respectifs (mer, ciel) fournissant un contexte sémantique fortement discriminant.

Le passage du Vanilla NN au Vanilla CNN réduit significativement ces confusions grâce à l'extraction de caractéristiques spatiales plus fines.

12 Anatomie d'un réseau convolutif

12.1 Couches de convolution

Le cœur d'un CNN est la couche de convolution, qui fait glisser un filtre (*kernel*) sur l'image pour produire une *feature map*. Un filtre 3×3 de détection de contours verticaux prend par exemple la forme :

$$K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix} \tag{1}$$

Les hyperparamsètres associés sont le *kernel size*, le *stride* (pas de déplacement) et le *padding* (gestion des bords).

12.2 Couches de pooling et régularisation

Le *Max Pooling* sous-échantillonne les feature maps en conservant la valeur maximale de chaque fenêtre, réduisant ainsi le volume de calcul et introduisant une invariance spatiale partielle.

La *Batch Normalization* normalise les activations par mini-lot, ce qui stabilise et accélère l'apprentissage. Le *Dropout* désactive aléatoirement un pourcentage de neurones à chaque étape d'entraînement, forçant le réseau à développer des représentations redondantes et moins sujettes à l'overfitting.

12.3 Fonctions d'activation

- **ReLU** : $\mathrm{ReLU}(x) = \max(0, x)$ — la plus utilisée, simple et efficace pour les couches intermédiaires.
- **Sigmoid** : sortie dans $[0, 1]$, adaptée à la classification binaire.
- **Softmax** : normalise les logits en distribution de probabilités pour la classification multi-classe.

L'encha^inement standard d'une architecture convolutive est :

$$\text{Image} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pooling} \rightarrow \text{Dense} \rightarrow \text{Softmax}$$

13 Pistes d'amélioration

Plusieurs leviers permettent d'améliorer les performances au-delà des architectures testées.

La **data augmentation** (rotations, retournements, zooms, décolorisation aléatoire) diversifie artificiellement le jeu d'entraînement et réduit l'écart train/test. Le **transfer learning** à partir d'un modèle pré-entraîné sur ImageNet (ResNet50, EfficientNet) permet d'atteindre rapidement de meilleures performances même avec peu de données spécifiques. L'ajout de couches de convolution supplémentaires enrichit la hiérarchie de caractéristiques apprise. Enfin, une recherche méthodique des hyperparamsètres (taux d'apprentissage, batch size, architecture) et l'utilisation de métriques complémentaires (recall, precision, F1-score) permettent un diagnostic plus complet des forces et faiblesses du modèle.

14 Synthèse des résultats

14.1 L'avantage décisif des CNN

La progression des performances de test de 35,47% (Simplest NN) à 80,59% (AlexNet) confirme que l'invariance par translation et l'extraction hiérarchique de caractéristiques propres aux CNN sont essentielles pour la vision par ordinateur. Les modèles entièrement denses atteignent un plafond structurel lié à leur incapacité à exploiter la topologie spatiale des images.

14.2 Overfitting et hyperparamsètres

Le couple Learning Rate = 0,001 / Batch Size = 256 constitue un point d'équilibre raisonnable. Un lot trop petit dégrade l'efficacité du calcul parallèle sur GPU ; un lot trop grand peut nuire à la généralisation en réduisant la stochasticité des mises à jour de gradient.

15 Explicabilité via Grad-CAM++

15.1 Principe

Grad-CAM++ (*Gradient-weighted Class Activation Mapping++*) produit une carte de chaleur (*heatmap*) en combinant les gradients de la classe prédite avec les activations des dernières couches convolutives. Les zones **rouges** indiquent les régions ayant le plus influencé la décision, les zones **bleues/sombres** signalent peu d'impact.

Par rapport à Grad-CAM classique, la version ++ améliore la localisation lorsque plusieurs instances d'un même objet sont présentes, grâce à une pondération plus fine des gradients.

Ses principales limites restent la résolution modeste des heatmaps (liée à la taille des feature maps), la dépendance à la structure du réseau, et le caractère qualitatif de l'interprétation.

15.2 Observations sur notre modèle

Appliqué au Vanilla CNN entraîné sur CIFAR-10, Grad-CAM++ confirme que les prédictions correctes activent bien les régions pertinentes de l'objet. Les erreurs, à l'inverse, révèlent que le modèle s'est fixé sur des éléments d'arrière-plan ou des textures non discriminantes, ce qui explique les confusions observées dans la matrice de confusion.

15.3 Autres méthodes d'explicabilité

- **LIME** : perturbe localement l'entrée et mesure l'impact sur la sortie.
- **SHAP** : fonde l'attribution sur la théorie des jeux coopératifs.
- **Integrated Gradients** : intègre les gradients le long d'un chemin entre une référence et l'entrée.
- **Occlusion Sensitivity** : masque systématiquement des régions de l'image et observe les variations de confiance.

16 Généralisation et enrichissement des datasets

16.1 Application sur d'autres datasets

Dataset	Classes	Époques	Précision Test
MNIST	10	20	98,95%
Fashion MNIST	10	20	90,93%
CIFAR-100	100	50	37,96%
CIFAR-10 augmenté	10	50	77,47%

Table 3: Précision en test pour chaque dataset (Vanilla CNN ré-entraîné)

MNIST atteint près de 99% grâce à sa simplicité intrinsèque ; Fashion MNIST est plus difficile du fait de la plus grande variabilité intra-classe des vêtements. CIFAR-100 plafonne à 37,96%, résultat cohérent face à la difficulté du problème (la précision aléatoire ne serait que de 1%). La data augmentation sur CIFAR-10 apporte un gain de +3,54% sans modification de l'architecture.

16.2 Enrichissement de CIFAR-10

Quatre images ont été générées par classe (40 au total) via RandomFlip, RandomRotation ± 15 et RandomZoom $\pm 15\%$. Soumises au modèle entraîné, elles n'obtiennent que 10% de précision (seul le cerf est correctement reconnu dans 4 cas sur 4). Les distorsions appliquées sur des images déjà très petites (32×32 pixels) déforment suffisamment les contours pour placer ces exemples hors de la distribution d'entraînement.

16.3 Enrichissement de Fashion MNIST

La même démarche appliquée à Fashion MNIST avec des transformations plus douces (± 10, $\pm 10\%$) donne 83,3% de précision (5 images sur 6 correctement classées). Les images en niveaux de gris sur fond uniforme sont nettement plus robustes aux transformations géométriques, confirmant que l'efficacité de la data augmentation dépend fortement des propriétés visuelles du dataset source.

17 Conclusion — TP Classification

Ce TP a mis en lumière la supériorité des architectures convolutives pour la classification d'images, tout en soulignant que cette puissance s'accompagne d'un risque d'overfitting croissant avec la complexité du modèle. Les outils d'explicabilité confirment que le réseau apprend véritablement des caractéristiques pertinentes pour les prédictions correctes. Les expériences de généralisation montrent que l'architecture Vanilla CNN est suffisamment flexible pour s'adapter à des domaines variés, sous réserve d'un ajustement minimal.

La voie naturelle de progression réside dans le *transfer learning* depuis des architectures profondes pré-entraînées (ResNet, EfficientNet) combiné à des techniques de régularisation avancées (weight decay, mixup, early stopping).

Part III

TP Couleur — Suivi de balle par couleur avec OpenCV

18 Introduction

Le tracking visuel par couleur est l'une des approches les plus simples et les plus efficaces pour suivre un objet mobile en temps réel. Ce troisième TP en explore les mécanismes à travers la détection et le suivi d'une balle colorée dans une vidéo, en utilisant la bibliothèque **OpenCV** et l'espace colorimétrique **HSV**. Ce type de système trouve des applications directes dans le pilotage autonome de drones, comme le montre l'intégration avec le module `autopylot_agent.py`.

Objectifs

- Maîtriser la segmentation par couleur dans l'espace HSV.
- Étendre un tracker mono-couleur au suivi simultané de plusieurs couleurs.
- Résoudre le problème de la présence de plusieurs objets de même couleur.
- Implémenter un apprentissage automatique de la plage HSV cible.
- Estimer la distance à l'objet par méthode empirique et par calibration caméra.

19 Question 1 — Analyse du code `balls_tracker.py`

19.1 Pipeline de traitement

Le code de base met en œuvre un pipeline en cinq étapes appliquées à chaque image de la vidéo :

1. Un **flou gaussien** atténue le bruit de capteur avant la segmentation.
2. La conversion **BGR** → **HSV** décompose l'information colorimétrique en teinte (indépendante de la luminosité), saturation et valeur.
3. Un **masque binaire** isole les pixels dont la teinte correspond au vert.
4. Le **centroïde** de la région masquée est calculé par les moments d'image (m_{00}, m_{10}, m_{01}).

5. Le centroïde est matérialisé par un cercle sur l'image originale et ses coordonnées sont retournées.

La fonction `track(image)` renvoie `(x, y)` si une balle est détectée, `(-1, -1)` en l'absence d'objet, et `None` sur appui sur Échap.

19.2 Utilisation comme capteur de position

Ce code constitue la brique de base d'un système de suivi visuel. Intégré dans une boucle de contrôle, il fournit à chaque frame les coordonnées de l'objet cible, directement exploitables par un contrôleur de drone ou tout autre système asservi.

20 Question 2 — Suivi multi-couleur

20.1 Plages HSV utilisées

L'espace HSV est préféré au BGR pour la segmentation par couleur car la teinte est codée indépendamment de la luminosité ambiante (plage 0–179 dans OpenCV).

Couleur	Lower HSV	Upper HSV
Vert	[40, 70, 70]	[80, 255, 255]
Bleu	[100, 100, 50]	[130, 255, 255]
Jaune	[20, 100, 100]	[35, 255, 255]
Rose	[140, 50, 50]	[170, 255, 255]

Table 4: Plages HSV retenues pour la détection des quatre couleurs

20.2 Code proposé

Listing 1: Tracker multi-couleur avec sélection de la plage HSV

```python
COLOR_RANGES = {
    'vert':  ([40,  70,  70],  [80,  255, 255]),
    'bleu':  ([100, 100, 50],  [130, 255, 255]),
    'jaune': ([20,  100, 100], [35,  255, 255]),
    'rose':  ([140, 50,  50],  [170, 255, 255]),
}

def track(image, color='vert'):
    lower, upper = COLOR_RANGES[color]
    blur  = cv2.GaussianBlur(image, (5, 5), 0)
    hsv   = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)
    mask  = cv2.inRange(hsv, np.array(lower), np.array(upper))
    bmask = cv2.GaussianBlur(mask, (5, 5), 0)
    moments = cv2.moments(bmask)
    m00 = moments['m00']
    ctr = (-1, -1)
    if m00 != 0:
        cx  = int(moments['m10'] / m00)
```

```
        cy  = int(moments['m01'] / m00)
        ctr = (cx, cy)
        cv2.circle(image, ctr, 10, (0, 0, 125), -1)
    cv2.imshow('BallTracker', image)
    if cv2.waitKey(1) & 0xFF == 27:
        return None
    return ctr
```

20.3 Résultats

Le tracker identifie correctement chaque balle sur `ball3.mp4` selon la couleur sélectionnée, le centroïde étant matérialisé par un point rouge.

21 Question 3 — Contrôle de drone (`autopylot_agent.py`)

21.1 Architecture du système

`autopylot_agent.py` exploite `greenball_tracker.py` comme capteur de position pour piloter un AR.Drone Parrot. À chaque frame, les coordonnées du centroïde sont converties en commandes de vol.

21.2 Calcul des erreurs et contrôle proportionnel

Pour une frame de dimensions (W, H), les erreurs de position par rapport au centre optique sont :

$$\Delta x = c_x - \frac{W}{2}, \qquad \Delta y = c_y - \frac{H}{2} \tag{2}$$

Ces écarts alimentent un **contrôleur proportionnel** générant les commandes de *roll* (translation latérale), *pitch* (translation avant/arrière) et *yaw* (rotation en lacet).

La **distance** est régulée indirectement via le moment m_{00} (surface du masque) : une surface importante signale une balle proche (le drone recule) ; une surface faible entraîne une avance du drone. L'objectif est de maintenir la balle au centre de l'image à une taille apparente constante.

22 Question 4 — Deux balles de même couleur

22.1 Comportement observé

En présence de deux balles de même couleur, les moments calculés sur le masque global correspondent au **barycentre** pondéré des deux régions détectées. Le centroïde résultant se situe dans le vide entre les deux objets et n'a aucune correspondance physique.

22.2 Conséquences et solution

Un drone cherchant à centrer ce point fant^ome oscillerait indéfiniment entre les deux balles, ce qui constitue un comportement instable voire dangereux en conditions réelles.

La solution adoptée par le code `track.py` (JJLewis) consiste à utiliser `cv2.findContours()` pour séparer les régions détectées, puis à ne conserver que le **contour de plus grande aire** : le drone suit alors systématiquement la balle la plus proche (donc la plus grande à l'écran).

23 Question 5 — Suivi du plus grand contour

Listing 2: Sélection du contour dominant avec opening morphologique

```
def track(image, color='vert'):
    lower, upper = COLOR_RANGES[color]
    blur = cv2.GaussianBlur(image, (5, 5), 0)
    hsv  = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, np.array(lower), np.array(upper))
    mask = cv2.erode(mask,  None, iterations=2)
    mask = cv2.dilate(mask, None, iterations=2)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    ctr = (-1, -1)
    if contours:
        biggest = max(contours, key=cv2.contourArea)
        if cv2.contourArea(biggest) > 500:
            M  = cv2.moments(biggest)
            cx = int(M['m10'] / M['m00'])
            cy = int(M['m01'] / M['m00'])
            ctr = (cx, cy)
            cv2.circle(image, ctr, 10, (0, 0, 125), -1)
    cv2.imshow('BallTracker', image)
    if cv2.waitKey(1) & 0xFF == 27:
        return None
    return ctr
```

L'érosion suivie de la dilatation (*opening morphologique*) supprime les petits artefacts dus au bruit avant la recherche de contours. Le seuil de 500 pixels carrés écarte les faux positifs résiduels.

24 Question 6 — Plus grosse balle rose (`ball4.mp4`)

L'appel `track(frame, color='rose')` avec le code précédent suffit : la sélection du plus grand contour garantit que le drone cible en permanence la balle rose la plus proche, même si d'autres objets roses sont présents dans la scène.

25 Questions 8 & 9 — Persistance du suivi

25.1 Le problème du saut de cible (Q8)

Lorsque deux balles de même couleur sont présentes et qu'elles changent de taille relative (l'une s'approche pendant que l'autre s'éloigne), le critère "plus grand contour" peut provoquer un basculement subit de la cible suivie d'une frame à l'autre. Ce comportement instable est inacceptable pour un système de contrôle.

Remarque : la démonstration vidéo n'a pas été réalisée faute de matériel (deux balles de même couleur et caméra). Le comportement décrit reste validé théoriquement.

25.2 Suivi par proximité spatiale (Q9)

La solution consiste à choisir, parmi tous les contours détectés, celui dont le **centroïde est le plus proche de la dernière position connue**. Cela assure la continuité temporelle du suivi en privilégiant la cohérence de trajectoire plutôt que la taille instantanée.

Listing 3: Suivi persistant par proximité spatiale

```
last_pos = None

def track_persistent(image, color='rose'):
    global last_pos
    lower, upper = COLOR_RANGES[color]
    blur = cv2.GaussianBlur(image, (5, 5), 0)
    hsv  = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, np.array(lower), np.array(upper))
    mask = cv2.erode(mask, None, iterations=2)
    mask = cv2.dilate(mask, None, iterations=2)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        if last_pos is None:
            chosen = max(contours, key=cv2.contourArea)
        else:
            def dist_to_last(c):
                M = cv2.moments(c)
                if M['m00'] == 0: return float('inf')
                cx = int(M['m10'] / M['m00'])
                cy = int(M['m01'] / M['m00'])
                return (cx - last_pos[0])**2 + (cy - last_pos[1])**2
            chosen = min(contours, key=dist_to_last)
        M = cv2.moments(chosen)
        if M['m00'] != 0:
            cx, cy   = int(M['m10']/M['m00']), int(M['m01']/M['m00'])
            last_pos = (cx, cy)
            cv2.circle(image, last_pos, 10, (0, 0, 125), -1)
    cv2.imshow('BallTracker', image)
    return last_pos if last_pos else (-1, -1)
```

26 Question 10 — Apprentissage automatique de la couleur

26.1 Principe

Plutôt que de saisir manuellement les bornes HSV, on laisse l'utilisateur placer l'objet cible dans un carré de sélection affiché au centre de l'image. Les valeurs HSV minimales et maximales des pixels de cette zone deviennent, après application d'une marge de tolérance, les bornes du masque de détection.

Listing 4: Calcul automatique de la plage HSV par ROI centrale

```
def learn_color_from_center(image, margin=20):
    h, w   = image.shape[:2]
    cx, cy = w // 2, h // 2
    roi    = image[cy-margin:cy+margin, cx-margin:cx+margin]
    cv2.rectangle(image, (cx-margin, cy-margin),
                  (cx+margin, cy+margin), (0, 255, 0), 2)
    cv2.imshow('BallTracker', image)
    hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
    pixels  = hsv_roi.reshape(-1, 3)
    lower   = np.clip(pixels.min(axis=0) - [10, 40, 40], 0, 255)
    upper   = np.clip(pixels.max(axis=0) + [10, 40, 40], 0, 255)
    return lower.astype(np.uint8), upper.astype(np.uint8)
```

27 Question 11 — Estimation de distance par interpolation

27.1 Modèle théorique

La perspective centrale relie le diamètre apparent D_{px} d'un objet de diamètre réel D_r à sa distance d :

$$d = \frac{D_r \times f}{D_{px}} \tag{3}$$

où f est la distance focale de la caméra exprimée en pixels.

27.2 Calibration empirique

En pratique, on mesure D_{px} à plusieurs distances connues, puis on ajuste une régression linéaire sur le couple $\left(\frac{1}{D_{px}}, d\right)$ pour estimer f. À titre indicatif, pour $D_r = 6$ cm et $f = 800$ px :

Distance réelle (cm)	Diamètre apparent (px)	Distance estimée (cm)
20	240	20,0
40	120	40,0
60	80	60,0
80	60	80,0
100	48	100,0

Table 5: Estimation de distance (loi de perspective centrale)

Remarque : la manipulation n'a pas été réalisée faute de matériel. La méthode décrite reste théoriquement valide.

28 Question 12 — Estimation de distance par calibration caméra

28.1 Modèle sténopé

Une calibration formelle avec échiquier (`cv2.calibrateCamera()`) fournit la matrice intrinsèque :

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \tag{4}$$

ainsi que les coefficients de distorsion $(k_1, k_2, p_1, p_2, k_3)$ modélisant les déformations radiales et tangentielles de l'objectif.

Après correction via `cv2.undistort()`, l'estimation de distance suit directement :

$$d = \frac{D_r \times f_x}{D_{px}} \tag{5}$$

28.2 Comparaison des deux approches

Critère	Q11 — Interpolation	Q12 — Calibration
Principe	Empirique (mesures directes)	Physique (modèle sténopé)
Précision	Bonne en zone centrale	Meilleure, surtout aux bords
Distorsion	Compensée implicitement	Corrigée explicitement
Mise en œuvre	Simple et rapide	Plus rigoureuse, nécessite un échiquier

Table 6: Comparaison des deux méthodes d'estimation de distance

29 Conclusion — TP Couleur

Ce TP a conduit une exploration progressive du tracking visuel par couleur, depuis la détection par moments jusqu'à la persistance du suivi en environnement multi-objets. L'espace HSV s'est confirmé comme le représentant le plus adapté à la segmentation par couleur en conditions variables d'éclairage. Le passage des moments aux contours résout le problème des occurrences multiples, tandis que le critère de proximité spatiale garantit la continuité de la cible suivie.

Les deux approches d'estimation de distance sont complémentaires : la première est déployable immédiatement ; la seconde, fondée sur la physique de la caméra, est plus robuste en dehors de la zone centrale de l'image. L'ajout d'un filtre de Kalman pour prédire la position entre deux frames constituerait l'amélioration naturelle pour améliorer la résistance aux occultations partielles.